

10th Feb 2024 | DA6401

⇒ Learning Algo.

- procedure to adjust model params. to min. a loss funcⁿ. Optimizers (in DL)

optimal params θ^* = arg min $L(\theta)$ $\rightarrow$ loss funcⁿ
all learnable θ params

Common ones are GD, SGD, Adam, RMSProp

- $w^{(1)} \dots w^{(L)}$ $\rightarrow$ layer index

- Parameter update $\rightarrow$ iterative process
 $\theta_{\text{new}} = \theta_{\text{old}} + \Delta\theta$ $\rightarrow$ random init.

- common problem $\rightarrow$ w/o proper scaling, we won't have control over params Δ , faster the network learns, stability of learning

$\Rightarrow \boxed{\theta_{\text{new}} = \theta_{\text{old}} + \eta \cdot \Delta\theta}$
 $\rightarrow$ learning rate

- entire setup

$\theta = [w, b]$, $\Delta\theta = [\Delta w, \Delta b]$

we can't update $\Delta\theta \rightarrow \theta$ leading to unstable learning $\rightarrow$ sometimes this can be very large.

$\theta_{\text{new}} = \theta + \eta \cdot \Delta\theta$

$0 < \eta < 1$ thus, have a controlled step size.

- how to derive the incremental update

$$L(\theta + \eta \cdot \Delta\theta)$$

Taylor expansion around θ involves
Hessian matrix (second derivatives)
 $\hookrightarrow H(\theta)$

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)(x-a)^2}{2!} + \dots$$

To min. the loss

$\eta^2, \eta^3 \rightarrow$ are
so small
that becomes
negligible

$$L(\theta + \eta \cdot \Delta\theta) < L(\theta)$$

$$\nabla_{\theta} L(\theta)^T \cdot \Delta\theta < 0$$

$$\boxed{\Delta\theta = -\nabla_{\theta} L(\theta)} \rightarrow \text{this gives the steepest descent direc}^n$$

- GD Update Rule

- $\nabla_{\theta} L(\theta)$ pts. in the direcⁿ of steepest inc. in the loss

- to min. the loss, we move in the opp. direcⁿ

$$\boxed{\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L(\theta_t)}$$

to move in downhill

θ_{t+1}

params at $t+1$

θ_t

params at t

η

learning rate

$\nabla_{\theta} L(\theta_t)$

gradient of loss w.r.t params

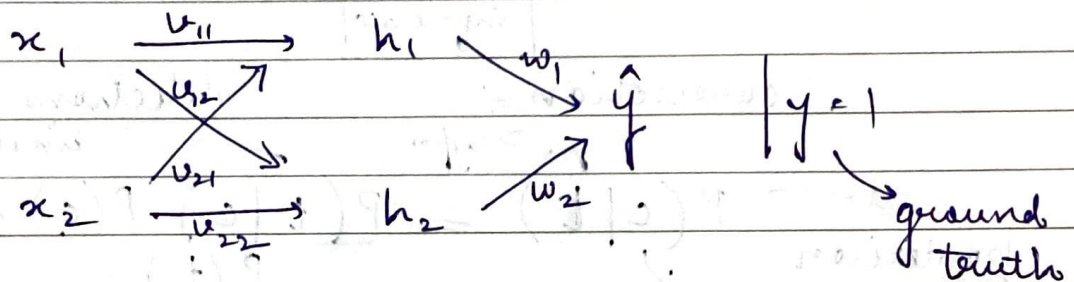
during convergence it becomes 0

batch=1 $\Rightarrow$ SGD $\rightarrow$ quicker, train (real-time)
 Mini-batch $\Rightarrow$ avg. the loss $\rightarrow$ noisy updates

- there are many net. init. algos, adaptive step-size.
- $w_{t+1}^{(l)} = w_t^{(l)} - \eta \frac{\partial L}{\partial w^{(l)}}$ $\rightarrow$ back-propagation algo.

$b_{t+1}^{(l)} = b_t^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$
 component wise

- complete learning process



- ① forward pass $h_1, h_2, \hat{y}$
- ② computation $L = \frac{1}{2} (\hat{y} - y)^2$ $\left[L = \frac{1}{2} \sum \ell(\hat{y}, y) \right]$ $\left[\text{1 batch} \right]$

- ③ backpropagation $\rightarrow$ gradients flow backwards through the network

- ④ update the params

e.g. $w_1 \leftarrow w_1 - \eta \frac{\partial L}{\partial w_1}$, etc. $\rightarrow$ iteration
 do for entire data & that too multiple times.

- ⑤ evaluate on validaⁿ set (not used for calculating the wt. updates).
 Get the L_{val} . Check for convergence & overfitting

SGD

$\Rightarrow$ Repeat until convergence

Stop when loss converges or reached max epochs.